

UNIT - III :Knowledge Representations: Knowledge Engineering, Representing structure in frames, Rules and data, Object-oriented systems, Natural language Semantics, Levels of representation

Knowledge Engineering

Knowledge Engineering is the systematic process of transforming domain-specific knowledge into a computable form for solving real-world problems efficiently. It involves capturing informal specifications, formalizing them into logical models, and using principles of knowledge representation to build practical systems.

Key Concepts and Examples

1. What is Knowledge Engineering?

- **Definition:** The application of logic and ontology to create computable models for solving domain-specific problems within constraints like budgets and deadlines.
 - **Example:** A knowledge engineer models a **traffic light system** where the light alternates between red and green automatically or can be manually controlled under special conditions.
-

2. Translating Informal Specifications

- **Challenge:** Informal, natural language descriptions must be translated into precise, computable formats.
 - **Example:**
 - **Informal Spec:** "A traffic light automatically turns red or green but has manual control."
 - **Formal Spec:**
 - Variables: `currentColor` (red/green), `autoSwitch` (on/off).
 - Rules:
 - If `autoSwitch` = on, green lasts `gg` seconds, then turns red.
 - If `autoSwitch` = off, manual inputs control the light.
-

3. Principles of Knowledge Representation

Knowledge representations serve multiple purposes in AI systems:

1. **Surrogate:** Models real systems computationally.
 - **Example:** Simulating a traffic light system with time-based changes.
 2. **Ontological Commitments:** Defines entities and relationships.
 - **Example:** Variables like `TrafficLight` or attributes like `currentColor`.
 3. **Intelligent Reasoning:** Enables logical deductions.
 - **Example:** If the light is red now, it will turn green in `rr` seconds.
 4. **Efficient Computation:** Supports algorithmic execution.
 - **Example:** A loop controlling the light's state transitions.
 5. **Human Expression:** Facilitates communication with domain experts.
 - **Example:** Simplified graphs or stylized English descriptions.
-

4. Approaches to Formalization

Declarative Approach

- Describes system behavior with rules and axioms.
- **Example:**
 - Initial state SS: Light is red, `autoSwitch = on`.
 - Deductive Rule: After `rr` seconds, the light turns green.

Procedural Approach

- Uses step-by-step instructions to simulate system operations.
 - **Example (Traffic Light):**

```
while autoSwitch == "on":  
    set currentColor to "red"  
    wait r seconds  
    set currentColor to "green"  
    wait g seconds
```
-

5. Ontological Commitments

Defines the system's structure through templates, frames, or schemas.

- **Example:** Traffic Light in a rule-based system like CLIPS:

```
(deftemplate trafficLight  
  (slot name (type SYMBOL))  
  (slot currentColor (allowed-values red green))  
  (slot redTime (type FLOAT))  
  (slot greenTime (type FLOAT))  
  (slot autoSwitch (allowed-values on off)))
```
-

6. Reasoning Strategies

Different strategies handle system operations:

1. **Procedural Loop:** Step-by-step execution for sequential systems.
 - Best for time-based processes (e.g., traffic light state transitions).
 2. **Logical Formulas:** Enables prediction or deduction.
 - **Example:** "If the light is red at t , it will turn green at $t+r$."
 3. **Forward-Chaining Rules:** Automatically updates the system when conditions are met.
-

7. Medium for Human Expression

To communicate with domain experts, knowledge must be represented intuitively:

- **Example:** Conceptual Graph (visual) or Stylized English:
 - "If a traffic light is red at time t , and the autoSwitch is on, it turns green at $t+r$."
-

Advantages of Different Approaches

1. **Procedural Approach:**
 - Best for processes with a natural sequence.
 - **Limitation:** Less suitable for parallel or complex relationships.
 2. **Declarative/Logical Approach:**
 - Excels at representing non-linear dependencies.
 - **Limitation:** Requires additional mechanisms for time or parallelism.
-

This summary distills key aspects of knowledge engineering with concrete examples, emphasizing how informal ideas are formalized into practical, computable systems.

Here's a more detailed explanation of **frames** and their role in knowledge representation, elaborated with examples from the provided text:

Frames: Representing Knowledge Structures

A **frame** is a structured way of representing knowledge about a specific, stereotypical situation or object. Think of a frame as a **template or schema** that describes what is typically true in a particular scenario or for an object.

For instance:

- A **living room frame** might include slots for furniture, such as a sofa, a TV, and lamps, along with their expected characteristics (e.g., the TV should be placed against a wall).
- A **birthday party frame** might include slots for decorations, a cake, and gifts, specifying expected actions like blowing out candles.

Structure of Frames

1. **Top Levels:** Represent facts that are always true about the scenario.
Example: In a "Traffic Light" frame, the fact that it cycles through colors (red, yellow, green) would be at the top level.
2. **Slots:** Represent attributes or variables specific to instances of the frame. These can be filled with specific data.
Example: For a "Traffic Light," slots might include:
 - **currentColor:** The current state of the light (e.g., "green").
 - **redTime:** Duration for which the light stays red.
 - **autoSwitch:** A boolean indicating if the light changes automatically.
3. **Conditions:** Specify rules for filling slots or constraints for their values.
 - A slot may require a specific type of value, such as "Duration" for **redTime** or "Color" for **currentColor**.

Example: Traffic Light Frame

Here is how a "Traffic Light" frame might look in a structured representation:

```
(defineType TrafficLight
  (supertype Object)
  (currentColor (type Color) (oneOf (red green yellow)))
  (redTime (type Duration))
  (greenTime (type Duration))
  (whenChanged (type PointInTime))
  (autoSwitch (type State) (oneOf (on off)))
)
```

- **Supertype:** "Object" indicates that a TrafficLight inherits general properties from the Object frame.
- **Slots:**
 - **currentColor:** Can only take values from "red," "green," or "yellow."
 - **redTime:** Duration for which the light stays red (e.g., "60 seconds").
 - **autoSwitch:** Indicates whether the traffic light switches automatically, with possible values "on" or "off."

Hierarchies and Inheritance

Frames support **hierarchies** through **supertypes** and **subtypes**:

- **Supertypes** represent general categories.
- **Subtypes** inherit properties and add specific details.

Example: Truck and TrailerTruck Frames

1. Truck Frame:

```
2. (defineType Truck
3.   (supertype Vehicle)
4.   (unloadedWt (type WtMeasure))
5.   (maxGrossWt (type WtMeasure))
6.   (cargoCapacity (type VolMeasure))
7.   (numberOfWheels (type Integer))
8. )
```

- **UnloadedWt** and **maxGrossWt** are weight measures.
- **cargoCapacity** is a volume measure (e.g., cubic meters).
- **numberOfWheels** must be an integer (e.g., 4, 6, or 18).

9. TrailerTruck Frame (Subtype of Truck):

```
10. (defineType TrailerTruck
11.   (supertype Truck)
12.   (hasPart (type Trailer))
13.   (numberOfWheels 18)
14. )
```

- Inherits all slots from the Truck frame.
- Adds a slot **hasPart** for a trailer.
- Restricts **numberOfWheels** to exactly 18.

Merged Representation:

The TrailerTruck frame effectively combines the inherited slots from the Truck frame, with its own specific details:

```
(defineType TrailerTruck
  (supertype Truck)
  (unloadedWt (type WtMeasure))
  (maxGrossWt (type WtMeasure))
  (cargoCapacity (type VolMeasure))
  (hasPart (type Trailer))
  (numberOfWheels 18)
)
```

Mapping Frames to Logic

Frames can be mapped to **logical representations**, such as:

- **Conceptual Graphs (CGs)**: Represent frames using graph structures.
- **Predicate Calculus**: Uses logical predicates and quantifiers.

Example: Mapping a TrafficLight Instance to Predicate Calculus

An instance `Blinky` of the `TrafficLight` frame:

```
(defineinstance Blinky
  (type TrafficLight)
  (currentColor green)
  (redTime 60 seconds)
  (greenTime 60 seconds)
  (whenChanged 0:00 hour)
  (autoSwitch on)
)
```

In predicate calculus:

```
(∃x: TrafficLight) (x = Blinky ∧
  currentColor(x, green) ∧
  redTime(x, <60, seconds>) ∧
  greenTime(x, <60, seconds>) ∧
  whenChanged(x, <0:00, hour>) ∧
  autoSwitch(x, on)).
```

- This representation expresses the same information in a logical form, where each slot is a predicate applied to the instance `Blinky`.
-

Multiple Inheritance

Frames allow **multiple inheritance**, where an instance or type can belong to multiple supertypes.

Example: Peterbilt Trailer Truck

The frame for a Peterbilt trailer truck:

```
(defineinstance ZF437TT
  (type Peterbilt)
  (type TrailerTruck)
  (manufacturedBy PeterbiltInc)
  (numberOfWheels 18)
)
```

- Inherits properties from both `Peterbilt` and `TrailerTruck`.
 - Combines features like `numberOfWheels = 18` and `manufacturedBy = PeterbiltInc`.
-

Advantages of Frames

1. **Organized Knowledge:** Frames group related knowledge into logical units, making it easier to manage.
2. **Inference:** Inheritance allows for efficient inference, as properties from supertypes propagate to subtypes or instances.
3. **Flexible Representation:** Frames can represent complex conditions and relationships using constraints in slots.

Limitations

1. Frames lack the expressive power to handle negations or implications (e.g., “There is no hippopotamus in this room”).
 2. Arithmetic computations or more complex logic often require external systems or procedural attachments.
-

Summary

Frames are a powerful tool for structuring and organizing knowledge in AI. They represent a scenario or object as a set of attributes (slots) with specific constraints and allow for hierarchical inheritance, making knowledge representation both efficient and intuitive. Examples like "TrafficLight" and "TrailerTruck" demonstrate how frames encapsulate knowledge in a reusable and logical format.

Summary: Rules and Data

During the 1970s, **universities pioneered expert systems**, while **Ted Codd developed relational databases at IBM**. Although expert systems and database systems differ in scale, their functionalities are converging as database systems handle more complex operations and expert systems are applied to larger data sets.

Key Differences:

- **Expert Systems:** Focus on executing repeated rules on **small data sets**.
- **Database Systems:** Execute short chains of rules on **large data sets**.

Common Logical Foundations: Both systems rely on the **existential-conjunctive (EC) subset of logic**, using two main inference rules:

- **Modus Ponens:** From p and $(p \rightarrow q)$, infer q (Forward Chaining).
- **Modus Tollens:** From $\neg q$ and $(q \rightarrow p)$, infer $\neg p$ (Backward Chaining).

Historical Developments and Key Systems

- **Planner (1971, MIT):** Combined **forward and backward chaining with relational databases**.

- **MYCIN (Stanford, 1976)**: A backward-chaining system for **diagnosing bacterial infections**.
- **OPS5 (Carnegie-Mellon)**: A forward-chaining system that became the foundation for commercial expert systems, such as **CLIPS**.
- **Prolog**: Developed in Europe, combined backward-chaining with logical operations, influencing database integration.

Integration of Systems and Query Techniques

- Systems like **Microplanner, Prolog, and SQL** use **backtracking** to solve queries:
 - Search relevant relations to find the desired data.
 - **Backtracking** tries different options if a goal cannot be met.

Optimization

- In **SQL databases**, optimizations like **indexing, hash coding, and goal reordering** improve query performance.
- In contrast, systems like **Prolog and Microplanner** require manual goal ordering or use **preprocessors** for optimization.

PLURALS AND SETS

- Handling plural expressions in **logic** is more cumbersome than in English.
- **Microplanner** uses **find operators** to find multiple matches.
- In **Prolog**, the **setof predicate** accumulates search results into a list.
- **SQL** includes built-in operations such as **GROUP BY and HAVING clauses** to efficiently manipulate large sets of data.

Overall, **expert systems and relational databases share a logical foundation**, with a convergence in their capabilities due to the integration of relational operations and logical inference techniques.

Explanation with Examples

The provided passage discusses how different knowledge representation and query languages—**SQL, Prolog, and CLIPS**—can express operations to solve queries while adhering to the same logical foundations but using distinct syntaxes and mechanisms. Let's break this down step by step with concrete examples.

Key Concepts

1. Knowledge Representation

Knowledge representation stores data about **objects**, their **attributes**, and their **relationships**. Examples include the relations:

- **Supports Relation**: supports(supporter, supportee)
This means **object A supports object B**.

- **Objects Relation:** objects(id, shape, color)
Represents **the properties of objects** (like shape, color, etc.).

Conceptual Graphs vs. Natural Language Queries

Conceptual graphs translate natural language questions into a graph form that closely mirrors the semantics of natural language.

1. SQL

In **SQL**, queries are formulated to select data from tables by filtering, grouping, and joining.

Example Query (English):

"Select all supportees from the supports and objects relations where each supportee has the shape 'block,' each supporter has the shape 'pyramid,' and group the answers by supportees, selecting only those supportees that have exactly three supporters."

SQL Translation:

```
CREATE VIEW sup_color AS
```

```
SELECT supporter, color
```

```
FROM supports, objects
```

```
WHERE supportee = id;
```

- This query defines a **view** sup_color that groups supporter and color by selecting data from the supports and objects tables.
- The WHERE clause ensures that only those rows where supportee matches id in the objects table are selected.

2. Prolog

Prolog uses a **rule-based logical syntax**, emphasizing backward chaining and pattern matching.

Prolog Rule for sup_color:

```
sup_color(S, C) :- supports(S, X), objects(X, *, C).
```

- In this Prolog rule:
 - sup_color(S, C) is **true** if S supports some object X, and X has a **color C**.
 - It searches for a combination of **S and X** that meets the conditions in the database.

Prolog Query to Find Supporters of Pyramid E:

```
setof(S, ind_supports(S, 'E'), L).
```

- Here, setof collects all supporters of pyramid E into a list L.

3. CLIPS

CLIPS is a **forward-chaining rule-based system** and is commonly used in **expert systems**. It emphasizes pattern matching and proactive updates.

Forward-Chaining Rule in CLIPS to maintain contains relationships:

```
(defrule checkForBoxSupporter
```

```
  (supports ?x ?y)
```

```
  (objects ?x box ?)
```

```
  =>
```

```
  (assert (contains ?x ?y)))
```

- In this CLIPS rule:
 - If ?x is a **box** and ?x supports some object ?y, it asserts that **?x contains ?y**.
 - This proactively updates the database's **contains relationships** to maintain constraints (e.g., boxes containing anything they support).

Key Comparisons

Feature	SQL	Prolog	CLIPS
Query Method	Declarative (Backward Chaining)	Logical (Backward Chaining)	Forward Chaining
Syntax	SELECT, JOIN, WHERE, GROUP BY	Rules with :- syntax	Pattern matching, defrule
Relation Handling	Views and Triggers	Rules with Recursion	Rule-based assertions
Optimization Focus	Query Optimizers	Not Optimized for Recursion	Rete Network for efficiency
Updating Database	Triggers	Pattern Matching Updates	Proactive Updates

4. Conceptual Graph Representation

In **Conceptual Graphs**, queries are expressed in a graph format close to natural language.

For Example:

Natural Language Query:

"Which blocks are supported by 3 pyramids?"

Conceptual Graph Translation:

- [Block: {*}?](Thme)~[Support]-7(Inst)-7[Pyramid: Col{*}@3]
- Here:
 - {*} represents **plurals**.
 - Thme and Inst are linguistic representations for **relationships**.

- The query asks to group **blocks collectively supported by exactly 3 pyramids**.

Practical Implications

- **Backward Chaining** (Prolog, SQL): Focuses on deducing results by exploring relationships backward through known facts.
- **Forward Chaining** (CLIPS): Updates the database by proactively ensuring relationships (e.g., boxes containing contents).
- Each system has trade-offs:
 - **SQL** excels at simple, declarative queries with large datasets.
 - **Prolog** uses recursion and pattern matching but lacks native support for some database operations.
 - **CLIPS** provides fast updates and real-time constraint checks.

Conclusion

- Different systems (**SQL, Prolog, CLIPS**) have distinct syntaxes but share **logical core principles**.
- Translating natural language queries into these systems requires **conceptual mapping and abstraction**.
- **Conceptual graphs** act as an intermediate representation, simplifying the transformation from natural language queries to lower-level languages like **SQL, Prolog, and CLIPS**.

Each system's choice depends on its specific goals and constraints, like performance, scalability, data relationships, and querying complexity.

Key Concepts with Text Examples

1. Object-Oriented Systems (OOPS) Overview

- OOPS combine **declarative** (specifying objects) and **procedural** (defining actions) styles.
- Instead of separating data and actions, they **integrate data and operations** into a single package.

Example: Truck Class

Imagine we have a class `Truck`. In Java, it's declared as:

Java Code:

```
public class Truck extends Vehicle {
```

```
private int unloadedWeight;
private int maxGrossWeight;
private int cargoCapacity;

// Getter methods to access private variables
public int getUnloadedWeight() { return unloadedWeight; }
public int getMaxGrossWeight() { return maxGrossWeight; }
public int getCargoCapacity() { return cargoCapacity; }
}
```

Text Explanation:

- Here, `Truck` **inherits properties** from its parent class `Vehicle`.
 - Data about weight and capacity is **encapsulated** in private variables.
 - External code must use **methods** (like `getUnloadedWeight()`) to access this information.
-

2. Encapsulation

Encapsulation restricts direct access to an object's internal data. Only class methods or subclasses can interact with these private variables.

Example: TrailerTruck Class

```
public class TrailerTruck extends Truck {
    private Trailer trailer;

    // Constructor to initialize the TrailerTruck
    public TrailerTruck() {
        trailer = new Trailer();
        System.out.println("A new TrailerTruck has been created.");
    }
}
```

Text Explanation:

- In this `TrailerTruck` class, we have a `trailer` object.
 - Other parts of the program cannot access the `trailer` directly.
 - The `constructor` initializes it and encapsulates its operations, following OOPS principles.
-

3. Mapping Frames to Objects

- In a **frame system**, a **slot** maps to an **instance variable** in OOPS.
- Inheritance in frames corresponds to **inheritance of properties and methods** in classes.

Example: Inheritance in Frames to Object-Oriented Code

- In the frame system, you have a parent `Vehicle` frame.
- In OOPS, `Truck` extends `Vehicle`, inheriting its properties.

Text Explanation:

- **Parent Frame (`Vehicle`):** Contains attributes like `speed`, `capacity`.
 - **Sub-frame (`Truck`):** Inherits these properties and adds new attributes like `cargoCapacity`.
-

4. Procedural Integration

OOPS require procedural code to perform actual actions.

Backward Chaining Example

- In parsing, backward chaining refers to converting grammar rules into **recursive calls**.

Text Example:

If your system needs to parse a sentence ("The truck is moving"), it uses **recursive descent parsing** to break down the sentence into individual parts (subject, verb, object).

Forward Chaining Example

- **Observer Pattern in an Inventory System**

```
Observable inventory = new Observable(); // The main inventory system
Observer warehouseManager = new Observer(); // Dependent warehouse system

// Whenever new products are added, observers get notified
inventory.addObserver(warehouseManager);
```

Text Explanation:

- In an **Observer Pattern**, every update in the `Observable` inventory **notifies dependent Observer systems automatically**.
-

5. Encapsulation Enhances Data Integrity

Encapsulation ensures that only the right parts of the code interact with an object's data.

Example: Simulating a Delivery System

- A `Truck` object's operations (e.g., loading, delivery) are determined by its internal methods rather than direct access by external programs.

Text Explanation:

- Encapsulation allows **operations like loading cargo or navigating routes** to remain hidden inside the class, ensuring data remains **safe and error-free**.
-

6. Logic-Based OOPS Systems

- In these systems, **procedures encode logical propositions** instead of explicitly stating them.

Example: Converting PowerLoom Frames to C++

- In PowerLoom, logical **frames are automatically converted into C++ declarations**, maintaining compatibility across programming systems.

Text Explanation:

- This ensures **automatic conversion of logical rules** into code, maintaining **efficiency and accuracy**.
-

7. Zooming In and Zooming Out

- Systems often have interactive displays allowing **detailed zoom-ins or broader zoom-outs** of processes and states.

Example: Birthday Party Visualization

- In a conceptual graph representing a birthday party:
 - **Zoom In:** Shows the process of cake cutting, guest interactions, and time intervals.
 - **Zoom Out:** Displays relationships between guests and candles in a concise visual format.

Text Explanation:

- Such visualizations help programmers and analysts **understand complex interactions efficiently** by focusing on relevant details.
-

8. Translation to Natural Language

Programmers often use **comments and documentation**, ensuring code readability.

Example

```
// Method to get the truck's cargo capacity
public int cargoCapacity() {
    return cargoCapacity;
}
```

Text Explanation:

- These **comments serve as a bridge between the code and human understanding**, ensuring clarity for **both programmers and non-technical users**.
-

Let me know if you'd like more specific examples or deeper clarifications! ☐

Simplified Key Points with Examples

1. Natural Language vs. Artificial Languages

- **Natural languages** (like English or Spanish) are used by humans to communicate.
 - Example: "She plays the piano."
 - **Artificial languages** (mathematics and programming) have limited syntax to avoid ambiguity.
 - Example: In math, $2 + 2 = 4$ is exact and unambiguous.
-

2. Background Knowledge in Language Understanding

- Understanding language requires **background knowledge** about the world and context.
 - Example: In a request about **the movie *Casablanca***:
 - The system must know:
 - *Casablanca* is a **movie**, not a city.
 - Humphrey Bogart is an **actor**, not just a name.
 - This knowledge is difficult for computers to acquire automatically.
-

3. Challenges in Language Processing

1. **Syntax Analysis** (Understanding Sentence Structure):
 - Computers can analyze words quickly but still struggle with sentence meaning.
 - **Example:** Sentence: *"Find me a programmer at Chase Manhattan Bank who knows graphics."*
 - The system must locate the right person, but they may not work in a department explicitly labeled **"graphics."**
 2. **Semantic Interpretation** (Understanding Word Meanings):
 - Resolving **ambiguous words** is difficult.
 - **Example:**
 - The word **"piano"** could mean:
 - A **musical instrument**
 - A **story in a building**
 - A **plane or surface**
 - Context helps decide the correct meaning.
-

4. Steps in Language Analysis (DANTE Example)

1. **Morphology** (Word Forms)
 - Break words into parts and identify their meanings.
 - **Example:** *"degli"* means **"of the"** (preposition + article).
 2. **Syntax** (Sentence Structure)
 - Analyze grammar to determine the relationships between words.
 - **Example:** In *"the association of industrialists"*, it splits into phrases showing **associations and parts**.
 3. **Semantics** (Concept Meaning)
 - Translate sentences into **conceptual graphs** that represent meanings and relationships.
 - **Example:**
 - Conceptual graph:
 - [Association] **contains** [Industrialists]
 - Meaning: **"The association includes industrialists."**
-

5. Resolving Ambiguities

- Many words have **multiple meanings**, which must be resolved with context and syntax.
 - **Example:**
 - **"Mezzogiorno"** means **"noon"** or **"the south of Italy"**.
 - Context determines whether it refers to **time or location**.
-

6. Inference & Knowledge Representation

- Conceptual-level representations allow **complex reasoning and inferences**.
 - **Example:** In a sentence about "**Raul Gardini becoming President**",
 - The system answers **who got the job** but also deduces:
 - **Who was President before or after the appointment?**
-

Key Takeaways

- **Natural languages** are rich but challenging for computers due to their ambiguity and vast background knowledge requirements.
- **Artificial languages** have strict rules but lack the expressive flexibility of natural languages.
- **Language processing relies on understanding syntax, semantics, and background knowledge**, which current AI systems still struggle to handle fully.

Simplified Key Points

1. Levels of Representation

Ron Brachman's Levels (1979)

Ron Brachman divided representation into **5 levels**, focusing on how knowledge is organized and implemented:

1. **Implementational Level**
 - Involves **data structures** used in programming.
 - Examples: **Atoms, pointers, lists**.
2. **Logical Level**
 - Uses **symbolic logic** (propositions, variables, quantifiers, etc.).
 - Example: Logical statements like "**All humans are mortal**".
3. **Epistemological Level**
 - Defines **concept types, subtypes, inheritance**, and relationships among concepts.
 - Example: A "**vehicle**" concept that includes "**car**," "**bus**," and "**motorcycle**."
4. **Conceptual Level**
 - Involves **semantic relations, objects, actions**, and **linguistic roles**.
 - Example: How **actions relate to objects**, like "**a car moving on a road**."
5. **Linguistic Level**
 - Represents **arbitrary concepts, words, and expressions** in **natural language**.
 - Example: **English phrases** like "**I am going to the store**."

2. Competence Levels (Rodney Brooks, 1986)

Rodney Brooks outlined **8 levels of competence** for mobile robots, showing increasing sophistication:

1. **Avoiding**
 - Robots **avoid obstacles**, both moving and stationary.
 2. **Wandering**
 - Robots move **randomly** while avoiding obstacles.
 3. **Exploring**
 - Robots search for **reachable areas** and head toward them.
 4. **Mapping**
 - Build a **map of the environment** and note routes.
 5. **Noticing**
 - Detect **environment changes**, like new obstacles or areas.
 6. **Reasoning**
 - Robots **identify objects**, reason about their relationships, and act on them.
 7. **Planning**
 - Create **plans to change the environment** in desired ways.
 8. **Anticipating**
 - Predict **the actions of other objects**, adjust plans proactively.
-

3. Design Levels (Zachman Framework)

Zachman's **5 levels** outline different perspectives of information representation in **systems architecture**:

1. **Scope** (Level 1)
 - Describes aspects **independent of computer representation**.
 - Focus: The **big picture of business operations**.
2. **Enterprise Model** (Level 2)
 - Still **independent of computer implementation**, but describes **business interactions**.
3. **System Model** (Level 3)
 - Descriptions are **implementation-independent**, selected by a **system analyst**.
4. **Technology Model** (Level 4)
 - Connects **data structures to physical representations**, linking **programming and operations**.
5. **Component Level** (Level 5)
 - Focuses on **specific implementation details**, where the connection to the outside world becomes **less apparent**.

Zachman's Matrix

- A matrix with **6 columns and 5 rows**, showing **30 perspectives on knowledge representation**.
 - It captures various views, such as **business goals, processes, and IT implementation details**.
-

Key Takeaways

1. **Representation Levels** (Brachman) focus on **abstract conceptual understanding and programming details**.
2. **Competence Levels** (Brooks) outline how **robotic systems evolve in complexity and functionality**.
3. **Design Levels** (Zachman) show how **systems are structured from business concepts down to technical implementation details**.